

BACKEND · PROYECTO FINAL «SOLUCIONES PARA EL CLENA»

Arquitectura

CLENA Digital — Modelo C4, Clean Architecture y decisiones técnicas

Equipo de origen	Samuel Lombana y Alan Fontalvo
Equipo de Backend	Samuel Ramos (Product Owner) · Sebastián Brito (Scrum Master) · Camilo del Toro (Development Team) · Jeison Acosta (Development Team)
Fecha	12 de noviembre de 2026
Versión	1.0

Índice

1. Propósito y alcance 3

2. Nivel 1 — Contexto 3

3. Nivel 2 — Contenedores 4

4. Nivel 3 — Componentes de la API 6

5. Nivel 4 — Código: un caso de uso y sus puertos 8

6. Estructura de carpetas del backend 10

7. Flujo de una petición y manejo de errores 11

8. Decisiones técnicas 12

9. Seguridad 13

10. Despliegue y calidad 14

1. Propósito y alcance

Este documento describe cómo está construido **CLENA Digital** con el modelo C4: el sistema en su contexto (nivel 1), sus contenedores (nivel 2), los componentes de la API en las cuatro capas de Clean Architecture (nivel 3) y el código de un caso de uso (nivel 4). Después explica la estructura de carpetas del backend, cómo se une un caso de uso con su repositorio, cómo viaja una petición y por qué se tomó cada decisión técnica.

Todo lo que aparece aquí corresponde al código de `main` al 12 de noviembre de 2026. Parte del documento `docs/arquitectura.md` que escribió Camilo del Toro, puesto al día con las funcionalidades de la Entrega 1.

2. Nivel 1 — Contexto

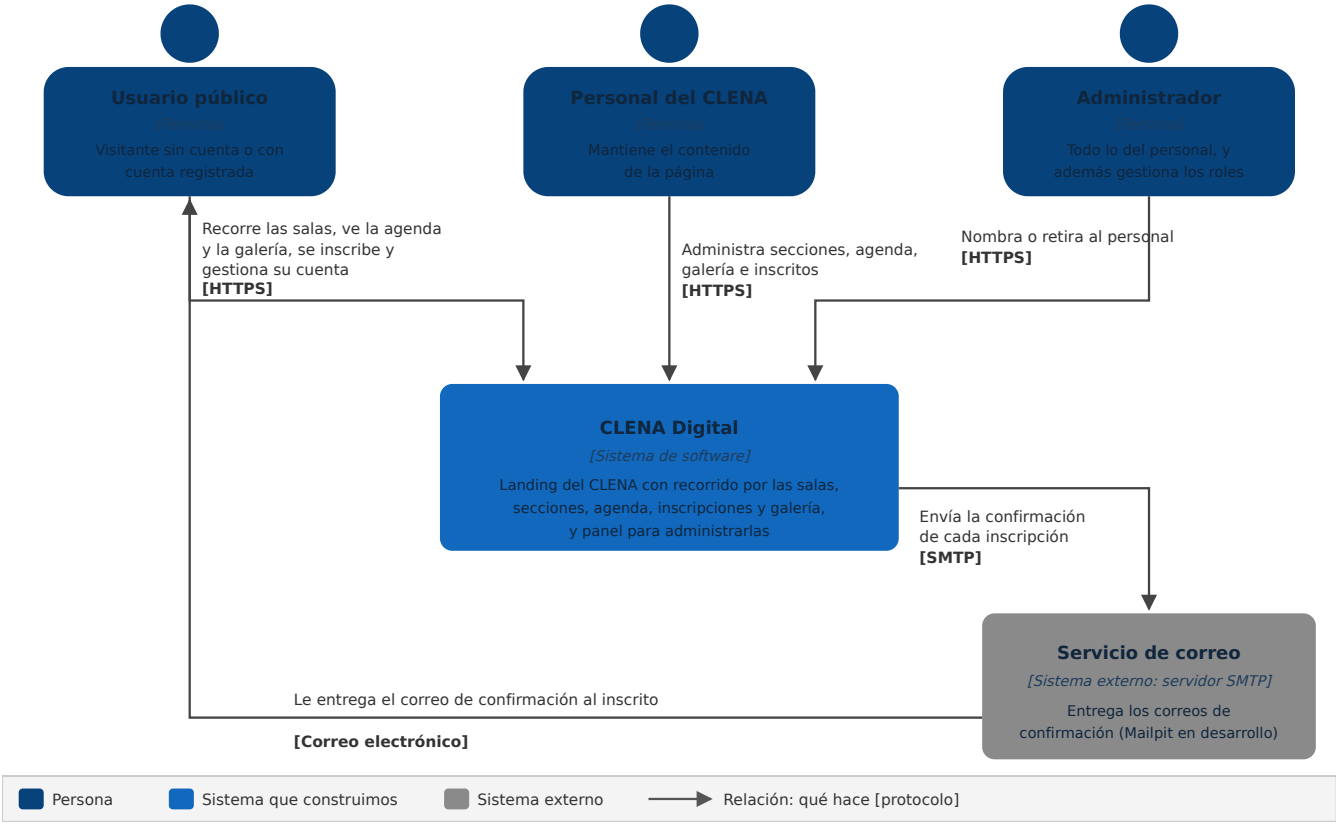


Figura 1. Diagrama de contexto (C4, nivel 1): CLENA Digital, sus tres tipos de usuario y el servicio de correo.

Elemento	Tipo	Qué hace con el sistema
Usuario público	Persona	Visitante sin cuenta o con cuenta registrada. Recorre las salas, ve la agenda y la galería, se inscribe a eventos y gestiona su cuenta.
Personal del CLENA	Persona	Administra las secciones de la portada, la agenda, la galería y ve los inscritos de cada evento.

Elemento	Tipo	Qué hace con el sistema
Administrador	Persona	Todo lo del personal, y además nombra o retira al personal del CLENA.
Servicio de correo	Sistema externo	Servidor SMTP que entrega la confirmación de cada inscripción. En desarrollo es Mailpit.

3. Nivel 2 — Contenedores

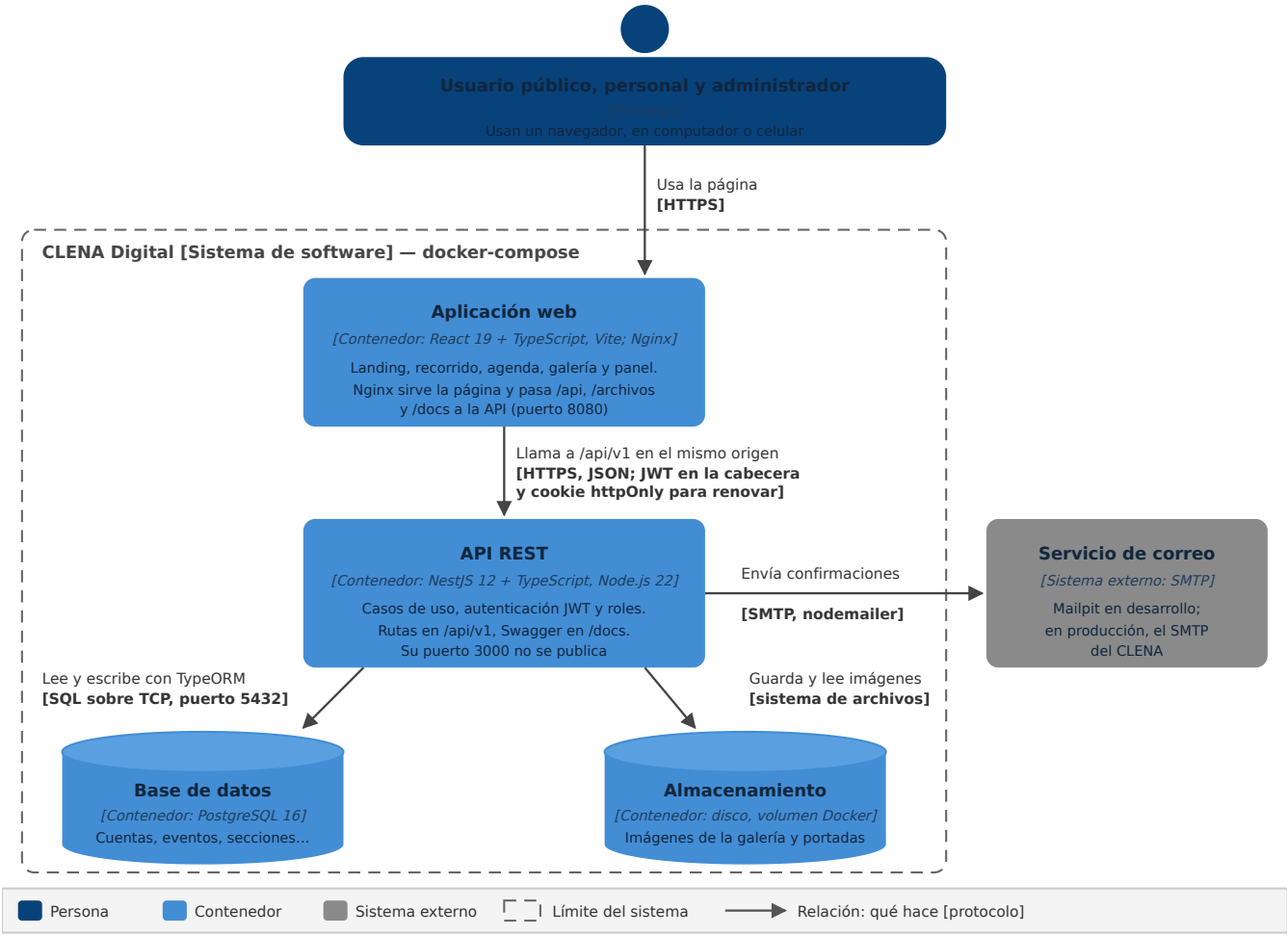


Figura 2. Diagrama de contenedores (C4, nivel 2): lo que levanta `docker compose up` y cómo se comunica cada parte.

Contenedor	Tecnología	Responsabilidad
Aplicación web	React 19 + TypeScript, Vite; servida por Nginx	Las pantallas públicas y el panel. Nginx sirve la página y pasa `/api`, `/archivos` y `/docs` a la API, así todo queda en el mismo origen (puerto 8080).
API REST	NestJS 12 + TypeScript sobre Node.js 22	Casos de uso, autenticación con JWT, roles y validación. Rutas bajo `/api/v1`, salud en `/api/v1/health` y Swagger en `/docs`. Su puerto 3000 no se publica: solo se llega por Nginx.

Contenedor	Tecnología	Responsabilidad
Base de datos	PostgreSQL 16	Cuentas, sesiones, secciones, eventos, inscripciones, galería y recorridos. El esquema cambia solo con migraciones versionadas.
Almacenamiento	Disco en un volumen de Docker (`/app/storage`)	Las imágenes de la galería y las portadas. En la base solo se guarda la clave de cada archivo.
Servicio de correo (externo)	SMTP; Mailpit en desarrollo	Recibe los correos de confirmación.

Conexión	Qué viaja	Protocolo
Navegador → Aplicación web	Páginas, scripts e imágenes	HTTPS
Aplicación web → API	Peticiones a `/api/v1` con el token de acceso en la cabecera `Authorization`, y la cookie `httpOnly` del token de renovación	HTTPS, JSON
API → Base de datos	Consultas de TypeORM	SQL sobre TCP (5432)
API → Almacenamiento	Lectura y escritura de imágenes	Sistema de archivos
API → Servicio de correo	Correos de confirmación con nodemailer	SMTP

4. Nivel 3 — Componentes de la API

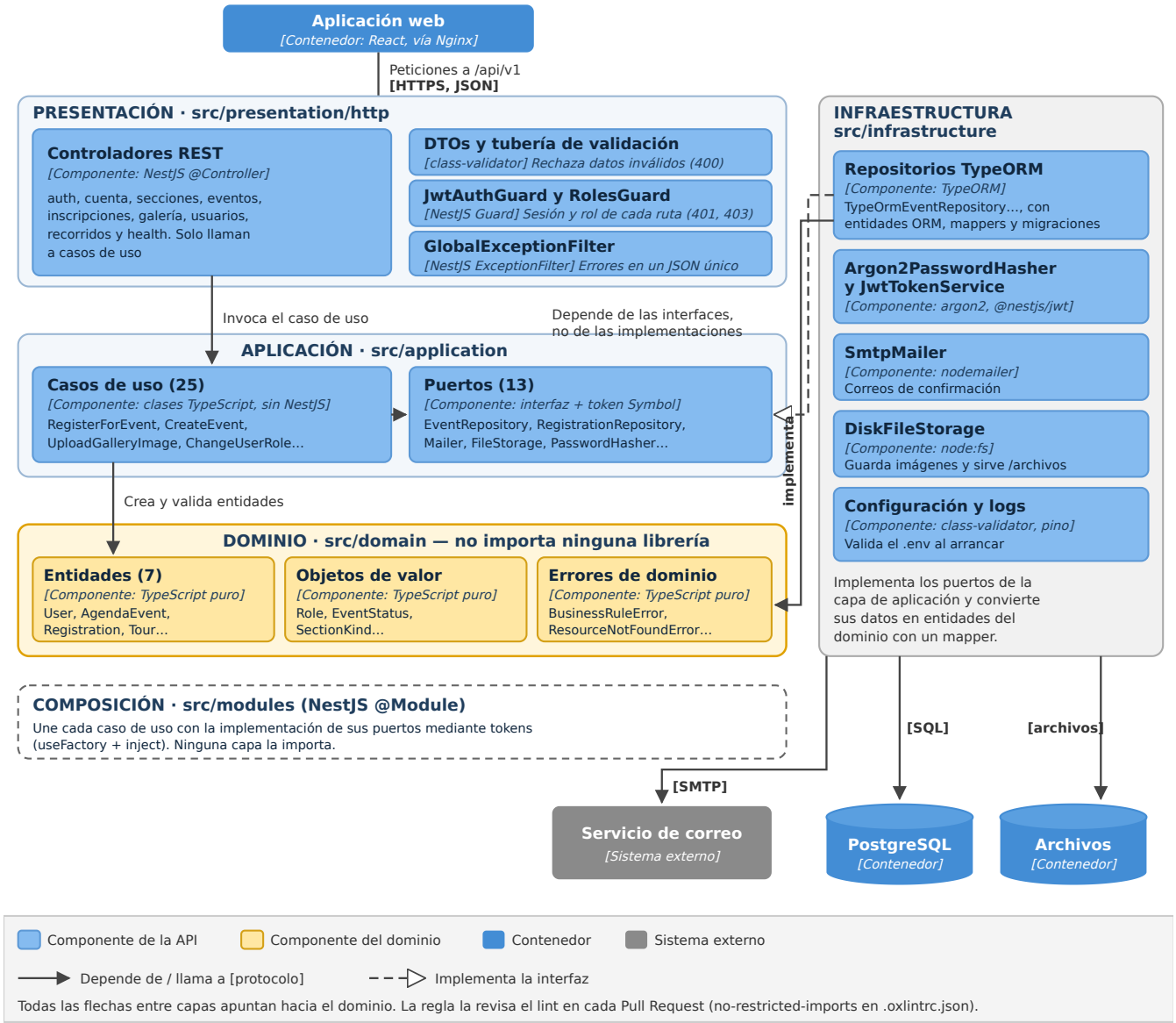


Figura 3. Diagrama de componentes de la API (C4, nivel 3), agrupados en las cuatro capas de Clean Architecture. Todas las dependencias apuntan hacia el dominio.

La API sigue Clean Architecture: el código se divide en cuatro capas y **las dependencias siempre apuntan hacia el dominio**. Ninguna capa interna conoce a una externa.

Capa	Carpeta	Contiene	Puede depender de
Dominio	`src/domain/`	Entidades, objetos de valor, reglas de negocio y errores de dominio. TypeScript puro.	Nada

Capa	Carpeta	Contiene	Puede depender de
Aplicación	`src/application/`	Un caso de uso por acción del negocio (25 en total) y los puertos que necesita: interfaces con su token.	Dominio
Infraestructura	`src/infrastructure/`	La implementación de los puertos: repositorios con TypeORM, entidades de persistencia, migraciones, hash de contraseñas, JWT, correo, almacenamiento, configuración y logs.	Dominio y aplicación
Presentación	`src/presentation/http/`	Controladores REST, DTOs con validación, guards de sesión y rol, y el filtro global de errores. Solo invoca casos de uso.	Aplicación y dominio
Composición	`src/modules/`	Módulos de NestJS que conectan cada puerto con su implementación. Ninguna capa la importa.	Todas

La regla se hace cumplir con el lint. `npm run lint` falla si alguien la rompe; está configurada con `no-restricted-imports` en `backend/.oxlintrc.json`, y el CI la corre en cada Pull Request:

Archivos en	No pueden importar
`src/domain/`	NestJS, TypeORM, `pg`, class-validator, class-transformer, pino, Express ni ninguna otra capa
`src/application/`	Esas mismas librerías, ni `infrastructure/`, `presentation/` o `modules/`
`src/presentation/`	TypeORM, `pg`, `infrastructure/` ni `modules/`

5. Nivel 4 — Código: un caso de uso y sus puertos

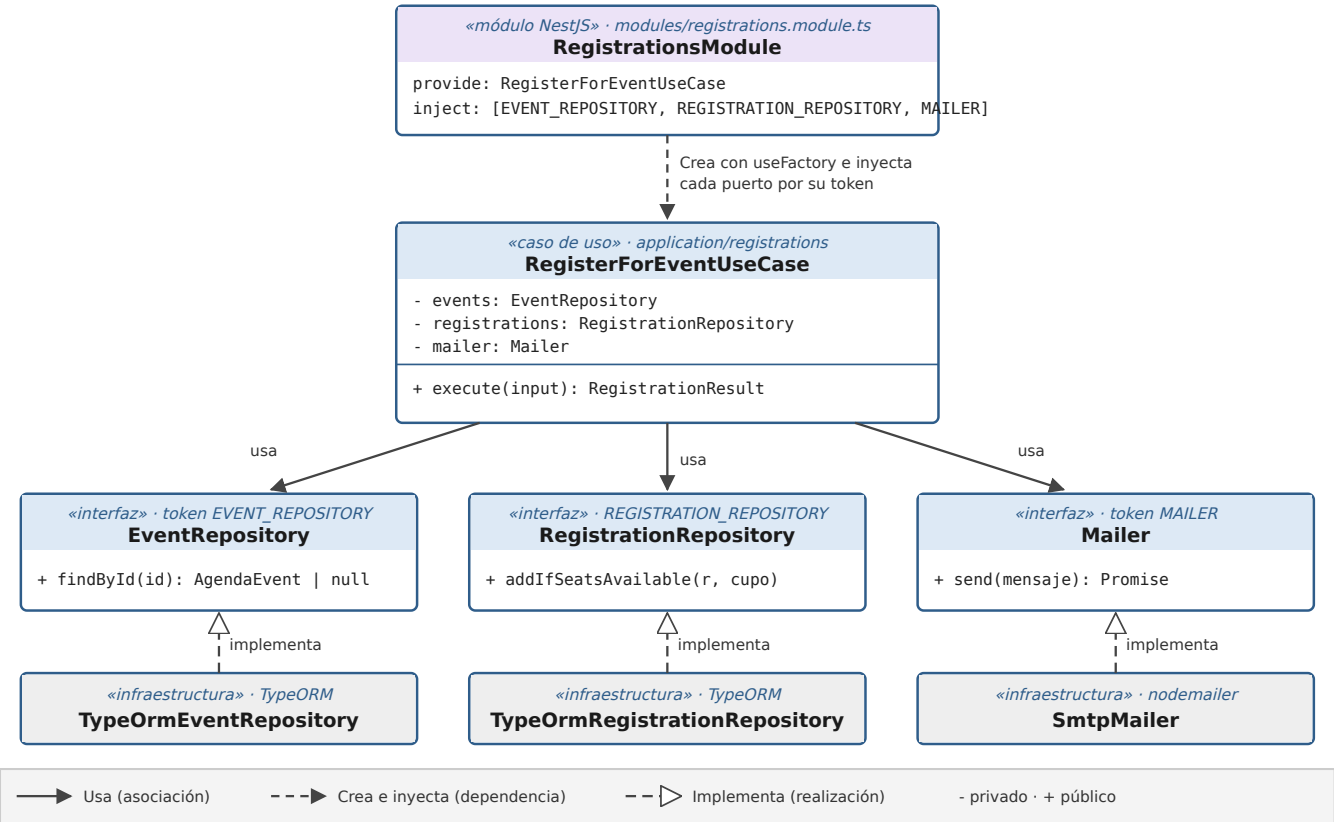


Figura 4. Diagrama de clases (C4, nivel 4) del caso de uso RegisterForEventUseCase: recibe sus puertos por token de inyección y no conoce sus implementaciones.

Los casos de uso son clases sin decoradores de NestJS: reciben sus dependencias por el constructor, tipadas con la interfaz del puerto. El módulo decide qué implementación se usa mediante un **token** (un `Symbol` exportado junto a la interfaz). Así el caso de uso se prueba con objetos falsos y la infraestructura se puede cambiar sin tocarlo.

El módulo que une el caso de uso con sus puertos

(`backend/src/modules/registrations.module.ts`):

```
@Module({
  imports: [EventsPersistenceModule, RegistrationsPersistenceModule, NotificationsModule],
  controllers: [RegistrationsController],
  providers: [
    {
      provide: RegisterForEventUseCase,
      inject: [EVENT_REPOSITORY, REGISTRATION_REPOSITORY, MAILER],
      useFactory: (events: EventRepository, registrations: RegistrationRepository, mailer: Mailer) =>
        new RegisterForEventUseCase(events, registrations, mailer),
    },
  ],
})
export class RegistrationsModule {}
```

El módulo que da la implementación de un puerto (`backend/src/modules/registrations-persistence.module.ts`). Está aparte para que las pruebas e2e lo cambien por un repositorio en memoria:

```
@Module({
  imports: [TypeOrmModule.forFeature([RegistrationOrmEntity])],
  providers: [
    {
      provide: REGISTRATION_REPOSITORY,
      inject: [getRepositoryToken(RegistrationOrmEntity)],
      useFactory: (repository: Repository<RegistrationOrmEntity>) =>
        new TypeOrmRegistrationRepository(repository),
    },
  ],
  exports: [REGISTRATION_REPOSITORY],
})
export class RegistrationsPersistenceModule {}
```

Las entidades de TypeORM (`*.orm-entity.ts`) se convierten en entidades de dominio con un mapper y nunca salen de la capa de infraestructura.

6. Estructura de carpetas del backend

```
backend/
├── src/
│   ├── domain/               Entidades, objetos de valor y errores, una carpeta por tema:
│   │                         users, sessions, sections, events, registrations, gallery, tours, errors
│   ├── application/          Casos de uso (*.use-case.ts) y sus puertos (ports/), por tema;
│   │                         common/ tiene la paginación compartida
│   ├── infrastructure/       Implementaciones por tema (typeorm-*.repository.ts, *.orm-entity.ts,
│   │   │                     *.mapper.ts) más:
│   │   ├── auth/             Argon2PasswordHasher y JwtTokenService
│   │   ├── config/           Validación de las variables de entorno
│   │   ├── database/         Conexión, migraciones y scripts (semilla de la galería, hacer admin)
│   │   ├── logging/          Configuración de pino
│   │   ├── notifications/    SmtpMailer
│   │   └── storage/          DiskFileStorage y la ruta /archivos
│   ├── presentation/http/    Un controlador y sus DTOs por tema, más auth/ (guards), errors/
│   │                         (filtro global), validation/ (tubería) y configure-app.ts
│   ├── modules/              Raíz de composición: <tema>.module.ts y <tema>-persistence.module.ts
│   ├── app.module.ts
│   └── main.ts
└── test/                     Pruebas e2e (*.e2e-spec.ts) y support/ con los repositorios en memoria
```

Tipo de archivo	Dónde va	Ejemplo
Entidad u objeto de valor	`src/domain/<tema>/`	`domain/events/agenda-event.ts`
Caso de uso	`src/application/<tema>/<accion>.use-case.ts`	`application/registrations/register-for-event.use-case.ts`
Puerto (interfaz y token)	`src/application/<tema>/ports/`	`application/registrations/ports/registration-repository.ts`
Repositorio, entidad ORM y mapper	`src/infrastructure/<tema>/`	`infrastructure/events/typeorm-event.repository.ts`
Migración	`src/infrastructure/database/migrations/`	`…-CreateEvents.ts`
Controlador y DTOs	`src/presentation/http/<tema>/`	`presentation/http/events/events.controller.ts`

Tipo de archivo	Dónde va	Ejemplo
Módulo de NestJS	`src/modules/`	`modules/events.module.ts`
Prueba unitaria / e2e	Junto al archivo (`*.spec.ts`) / `test/*.e2e-spec.ts`	`agenda-event.spec.ts`, `eventos.e2e-spec.ts`

Idioma: clases, archivos, carpetas y variables en inglés (`RegisterForEventUseCase`); campos del JSON, códigos de error, mensajes y comentarios en español (`estado`, `CUPO_NO_VALIDO`).

7. Flujo de una petición y manejo de errores

Así se atiende `POST /api/v1/eventos/:id/inscripciones`:

1. Nginx recibe la petición y la pasa a la API.
2. El límite de intentos (`@nestjs/throttler`) acepta hasta 10 por minuto por visitante.
3. La tubería de validación revisa el DTO: si falta un campo o sobra uno, responde 400 sin llegar al caso de uso.
4. `RegistrationsController` llama a `RegisterForEventUseCase.execute()`.
5. El caso de uso busca el evento por el puerto `EventRepository` y revisa que esté publicado.
6. Crea la entidad `Registration`, que exige el consentimiento del aviso de privacidad.
7. `RegistrationRepository.addIfSeatsAvailable()` guarda la inscripción en una transacción que bloquea la fila del evento, para que dos personas no tomen el último cupo.
8. El puerto `Mailer` envía la confirmación. Si el correo falla, la inscripción no se pierde.
9. El controlador responde 201 con los datos de la inscripción.

Toda excepción termina en `GlobalExceptionFilter`, que responde siempre con el mismo formato:

```
{ "estado": 409, "codigo": "EVENTO_SIN_CUPOS", "mensaje": "El evento ya no tiene cupos disponibles",
  "ruta": "/api/v1/eventos/…/inscripciones", "fecha": "2026-11-12T12:00:00.000Z" }
```

Origen del error	Estado HTTP	Código
DTO inválido	400	`VALIDACION`, con `detalles` por campo
JSON mal formado	400	`JSON_INVALIDO`
`AuthenticationError` (dominio) o sin token	401	`NO_AUTENTICADO` o uno propio
`AccessDeniedError` (dominio) o rol insuficiente	403	`ACCESO_DENEGADO`
`ResourceNotFoundError` (dominio)	404	`RECURSO_NO_ENCONTRADO`

Origen del error	Estado HTTP	Código
`ConflictError` (dominio)	409	`CONFLICTO` o uno propio, como `YA_INSCRITO`
`BusinessRuleError` (dominio)	422	`REGLA_DE_NEGOCIO` o uno propio, como `CUPO_NO_VALIDO`
Demasiados intentos	429	`DEMASIADAS_SOLICITUDES`
Cualquier otro error	500	`ERROR_INTERNO`, sin detalles internos; se registra en los logs

El dominio lanza errores que no saben nada de HTTP; la traducción a códigos de estado ocurre solo en la capa de presentación.

8. Decisiones técnicas

El stack (NestJS, React, PostgreSQL y Docker) es obligatorio. Estas son las decisiones que tomó el equipo:

Decisión	Alternativas	Por qué
ORM: TypeORM	Prisma	Sus entidades son clases con decoradores, que encajan con NestJS y con el patrón de repositorio por puerto. Genera migraciones a partir de las entidades.
Migraciones con `synchronize: false`	`synchronize: true`	El esquema solo cambia con migraciones versionadas, que quedan en el historial y se pueden revertir. Las migraciones también cargan las secciones y los eventos de ejemplo.
Validación: class-validator con una tubería global	Validar a mano en cada controlador	Toda entrada se valida igual. Rechaza campos que no están en el DTO (`forbidNonWhitelisted`), lo que evita que un cliente cambie datos que no debe.
Logger: pino (`nestjs-pino`)	Logger por defecto de NestJS	Logs en JSON que se pueden filtrar. Cada petición queda con método, ruta, estado y duración; se ocultan las cabeceras `authorization` y `cookie` y el token de renovación.
Librería de componentes del frontend: ninguna, CSS propio	Material UI, Chakra, Bootstrap	Las pantallas son pocas y la accesibilidad es requisito: con CSS propio se controla el contraste, los botones de 44 px y el texto al 200 %. El visor 360° usa Photo Sphere Viewer.
Casos de uso sin decoradores, creados con `useFactory`	`@Injectable()`	Mantiene la capa de aplicación libre de NestJS y fácil de probar.

Decisión	Alternativas	Por qué
Contraseñas con argon2	bcrypt	Es el algoritmo recomendado hoy para guardar contraseñas.
Sesión: token de acceso de 15 minutos en memoria y token de renovación en cookie `httpOnly` que rota	Guardar el token en `localStorage`	Un script malicioso no puede leer la cookie, y el token robado de una pestaña vence pronto.
Correo con nodemailer, detrás del puerto `Mailer`	Un servicio de correo con SDK propio	Funciona con cualquier SMTP; en desarrollo se usa Mailpit.
Imágenes en disco, detrás del puerto `FileStorage`	Un servicio de objetos (S3)	Es suficiente para el alcance y no tiene costo. Si cambia, solo se reemplaza la implementación del puerto.
Validación de las variables de entorno al arrancar	Leer `process.env` donde se necesite	Si falta una variable, la API no arranca y dice cuál es.
Swagger generado desde el código	Documentación a mano	`/docs` siempre coincide con la API real.
Vitest en el backend y el frontend; **Testing Library** en el frontend	Jest	Viene con NestJS 12 y Vite, y funciona con módulos ES.
oxlint en el backend	ESLint	Viene con NestJS 12 y permite `no-restricted-imports` por carpeta, que hace cumplir la regla de dependencia.

9. Seguridad

Tema	Cómo se resuelve
Autenticación	JWT: token de acceso de 15 minutos y token de renovación de 7 días que rota en cada uso; si se reusa uno viejo, se anulan todas las sesiones de la cuenta.
Contraseñas	Solo se guarda su hash con argon2.
Autorización	`JwtAuthGuard` y `RolesGuard` en cada ruta, con tres roles: administrador (`admin`), personal del CLENA (`staff`) y usuario público (`public`).
Datos personales (Ley 1581 de 2012)	Aviso de privacidad y consentimiento al registrarse y al inscribirse, con su fecha y versión; eliminación de la cuenta y de sus datos.
Fuerza bruta	Máximo 10 intentos por minuto en el inicio de sesión, el registro y la inscripción.
Archivos	Solo JPG o PNG, revisados por su contenido, de máximo 10 MB.
Secretos	Solo por variables de entorno; en el repositorio solo está `.env.example`.
Errores	Los inesperados responden 500 sin detalles internos.

Tema	Cómo se resuelve
Exposición	La API no publica su puerto: todo entra por Nginx.

10. Despliegue y calidad

Despliegue. `docker compose up --build` levanta cuatro servicios: PostgreSQL, la API (aplica las migraciones al arrancar), Nginx con la aplicación web y Mailpit, con dos volúmenes: uno para la base de datos y otro para las imágenes. La página queda en `http://localhost:8080` y Mailpit en `http://localhost:8025`.

Requisito del mismo sitio. La cookie del token de renovación es `SameSite=Strict`: el navegador solo la envía si la página y la API están en el mismo sitio. Por eso Nginx sirve las dos en el mismo origen. En un despliegue con dominios distintos la sesión se perdería al recargar la página.

Calidad. En cada Pull Request, GitHub Actions corre en el backend el lint, las pruebas unitarias con cobertura (falla por debajo del 70 % en dominio y aplicación), las pruebas e2e y el build; y en el frontend, el lint, las pruebas y el build. `main` está protegida: sin push directo, con un PR aprobado por otro integrante y el CI en verde.